```
import pandas as pd

def load_data(filepath):
    df = pd.read_csv(filepath)

    # Fill missing values (same as your preprocessing)
    for c in df.columns:
        if df[c].dtype == "object":
            df[c] = df[c].fillna(df[c].mode()[0])
        else:
            df[c] = df[c].fillna(df[c].median())
    return df
from sklearn.model_selection import train_test_split

def preprocess_data(df, target):
    cat_cols = [c for c in df.select_dtypes(include="object").columns if c != target]
    df_enc = pd.get_dummies(df, columns=cat_cols, drop_first=True)

    X = df_enc.drop(columns=[target])
    y = df_enc[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    return X_train, X_test, y_train, y_test
    import lightgbm as lgb

def train_quantile_models(X_train, y_train):
    qs = [0.1, 0.5, 0.9]
    models = {}

    for q in qs:
        model = lgb.LGBMRegressor(
            objective="quantile",
            alpha=q,
            n_estimators=600,
            learning_rate=0.05,
            num_leaves=31,
            random_state=42
        )
        model.fit(X_train, y_train)
        models[q] = model

    return models
def make_predictions(models, X_test):
    y_lo = models[0.1].predict(X_test)
    y_med = models[0.5].predict(X_test)
    y_hi = models[0.9].predict(X_test)

    return y_lo, y_med, y_hi
```

```
!pip install pyngrok streamlit lightgbm shap seaborn properscoring
```

```
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (8.3.1)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit) (3.1.4
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (26.0)
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (11.3.0)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.1-py2.py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: protobuf<8,>=3.20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.29.6)
```

```
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->streaml
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.(
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7->
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seabo
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seal
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seal
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seabo
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<4,>=1.4.0->streamlit) (202
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<4,>=1.4.0->streamlit) (2(
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->st
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (3.:
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.(
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython!=3.:
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->altair!=5.4.0,!=5.4.1,
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6
Downloading pyngrok-8.0.0-py3-none-any.whl (24 kB)
Downloading streamlit-1.56.0-py3-none-any.whl (9.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 90.1 MB/s eta 0:00:00
Downloading properscoring-0.1-py2.py3-none-any.whl (23 kB)
Downloading pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 127.8 MB/s eta 0:00:00
Installing collected packages: pyngrok, pydeck, properscoring, streamlit
Successfully installed properscoring-0.1 pydeck-0.9.1 pyngrok-8.0.0 streamlit-1.56.0
```

```
%%writefile app.py
import streamlit as st
import pandas as pd
import numpy as np
import lightgbm as lgb
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from properscoring import crps_ensemble
from scipy.stats import genextreme as gev

st.set_page_config(page_title="Interactive Crop Yield Dashboard", layout="wide")

st.title("🌾 INTERACTIVE CROP YIELD & RISK DASHBOARD")

uploaded_file = st.file_uploader("Upload your dataset (CSV)", type=["csv"])

if uploaded_file is not None:
    data = pd.read_csv(uploaded_file)
    st.write("### Dataset Preview")
    st.dataframe(data.head())

    target = "Crop_Yield_MT_per_HA"

    # ---------- PREPROCESSING ----------
    for c in data.columns:
        if data[c].dtype == "object":
            data[c] = data[c].fillna(data[c].mode()[0])
        else:
            data[c] = data[c].fillna(data[c].median())

    cat_cols = [c for c in data.select_dtypes(include="object").columns if c != target]
    df_enc = pd.get_dummies(data, columns=cat_cols, drop_first=True)

    X = df_enc.drop(columns=[target])
    y = df_enc[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
```

```
# ========== 🔹 INTERACTIVE SLIDERS (SIDEBAR) ==========
st.sidebar.header("🎛️ Interactive Input Controls")

slider_inputs = {}

for col in X.columns:
    if np.issubdtype(X[col].dtype, np.number):
        slider_inputs[col] = st.sidebar.slider(
            col,
            float(X[col].min()),
            float(X[col].max()),
            float(X[col].median())
        )

# ---- CREATE USER INPUT DATAFRAME ----
user_input_df = pd.DataFrame([slider_inputs])

# ---- ALIGN WITH TRAINING FEATURES (CRITICAL FIX) ----
user_input_df = user_input_df.reindex(columns=X_train.columns, fill_value=0)

# ---------- TRAIN MODELS ----------
if st.button("🚀 Train & Predict"):
    st.info("Training models... please wait")

    # ===== QUANTILE MODELS =====
    qs = [0.1, 0.5, 0.9]
    models = {}

    for q in qs:
        model = lgb.LGBMRegressor(
            objective="quantile",
            alpha=q,
            n_estimators=600,
            learning_rate=0.05,
            num_leaves=31,
            random_state=42
        )
        model.fit(X_train, y_train)
        models[q] = model

    y_lo = models[0.1].predict(X_test)
    y_med = models[0.5].predict(X_test)
    y_hi = models[0.9].predict(X_test)

    # ===== METRICS =====
    rmse = np.sqrt(mean_squared_error(y_test, y_med))
    r2 = r2_score(y_test, y_med)
    coverage = np.mean((y_test >= y_lo) & (y_test <= y_hi)) * 100
    interval_width = np.mean(y_hi - y_lo)

    st.success("Models Trained Successfully!")

    c1, c2, c3, c4 = st.columns(4)
    c1.metric("RMSE", f"{rmse:.3f}")
    c2.metric("R²", f"{r2:.3f}")
    c3.metric("Coverage %", f"{coverage:.1f}")
    c4.metric("Interval Width", f"{interval_width:.3f}")

    # ========== 🔹 REAL-TIME SINGLE PREDICTION ==========
    st.write("## 🌾 Real-Time Yield Prediction (Your Inputs)")

    user_pred_lo = models[0.1].predict(user_input_df)[0]
    user_pred_med = models[0.5].predict(user_input_df)[0]
    user_pred_hi = models[0.9].predict(user_input_df)[0]

    st.metric("Predicted Yield (Median)", f"{user_pred_med:.2f} MT/ha")
    st.write(f"Uncertainty Range: **{user_pred_lo:.2f} – {user_pred_hi:.2f} MT/ha**")

    # ===== PLOT 1: ACTUAL vs PREDICTED =====
    st.write("## Predicted vs Actual")

    fig1, ax1 = plt.subplots(figsize=(7,7))
    ax1.scatter(y_test, y_med, alpha=0.6)
    ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    ax1.set_xlabel("Actual Yield")
    ax1.set_ylabel("Predicted Yield")
    ax1.set_title("Actual vs Predicted")
```

```
        st.pyplot(fig1)

        # ===== UNCERTAINTY BAND =====
        st.write("## Uncertainty Band (10–90%)")

        fig2, ax2 = plt.subplots(figsize=(7,7))
        ax2.scatter(y_test, y_med, alpha=0.6)
        ax2.fill_between(np.sort(y_test),
                         np.sort(y_lo),
                         np.sort(y_hi),
                         alpha=0.3)
        ax2.set_title("Prediction with Uncertainty Interval")
        st.pyplot(fig2)

        # ===== EVT TAIL RISK =====
        st.write("## Extreme Tail Risk (EVT)")

        residuals = y_test - y_med
        losses = -residuals[residuals < 0]

        shape, loc, scale = gev.fit(losses)
        extreme_loss = gev.ppf(0.99, shape, loc=loc, scale=scale)

        x = np.linspace(0, max(losses.max(), extreme_loss)*1.1, 300)
        pdf = gev.pdf(x, shape, loc=loc, scale=scale)

        fig3, ax3 = plt.subplots(figsize=(7,7))
        ax3.plot(x, pdf)
        ax3.axvline(extreme_loss, linestyle="--", color="red")
        ax3.set_title("EVT Tail Risk Curve")
        st.pyplot(fig3)

        st.write(f"99th Percentile Loss: {extreme_loss:.2f} MT/ha")

        # ===== FEATURE IMPORTANCE =====
        st.write("## Top 10 Important Features")

        imp_gain = pd.DataFrame({
            "Feature": X_train.columns,
            "Importance": models[0.5].feature_importances_
        }).sort_values("Importance", ascending=False).head(10)

        fig5, ax5 = plt.subplots(figsize=(8,6))
        sns.barplot(data=imp_gain, y="Feature", x="Importance", ax=ax5)
        st.pyplot(fig5)

        # ===== SHAP EXPLAINABILITY (for your input) =====
        st.write("## SHAP Explanation for Your Input")

        explainer = shap.TreeExplainer(models[0.5])
        shap_values_user = explainer(user_input_df)

        fig6, ax6 = plt.subplots()
        shap.waterfall_plot(shap_values_user[0], show=False)
        st.pyplot(fig6)
```

```
Writing app.py
```

```
from pyngrok import ngrok

ngrok.set_auth_token("39Vus0BywcqceMkGWg7ekI4vEGL_28fSpuaULXJf4E4B5H2yi")
```

```
!streamlit run app.py --server.port 8501 --server.address 0.0.0.0 &>/content/logs.txt &
```

```
from pyngrok import ngrok

# Close any old tunnels (if they exist)
ngrok.kill()

# Start a fresh tunnel on Streamlit port
public_url = ngrok.connect(8501)
public_url
```

```
<NgrokTunnel: "https://inculpable-palma-selenic.ngrok-free.dev" -> "http://localhost:8501">
```

```
%%writefile app.py
import streamlit as st
import pandas as pd
import numpy as np
import lightgbm as lgb
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from properscoring import crps_ensemble
from scipy.stats import genextreme as gev

st.set_page_config(page_title="Integrated Crop Yield Dashboard", layout="wide")

st.title("🌾 FULL INTEGRATED LIVE DASHBOARD (ALL PLOTS IN ONE FRAME)")

uploaded_file = st.file_uploader("Upload your dataset (CSV)", type=["csv"])

if uploaded_file is not None:
    data = pd.read_csv(uploaded_file)
    st.write("### Dataset Preview")
    st.dataframe(data.head())

    target = "Crop_Yield_MT_per_HA"

    # ---------- PREPROCESSING ----------
    for c in data.columns:
        if data[c].dtype == "object":
            data[c] = data[c].fillna(data[c].mode()[0])
        else:
            data[c] = data[c].fillna(data[c].median())

    cat_cols = [c for c in data.select_dtypes(include="object").columns if c != target]
    df_enc = pd.get_dummies(data, columns=cat_cols, drop_first=True)

    X = df_enc.drop(columns=[target])
    y = df_enc[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # ========== 🔹 INTERACTIVE SLIDERS (SIDEBAR) ==========
    st.sidebar.header("🎛 Interactive Input Controls")

    slider_inputs = {}

    for col in X.columns:
        if np.issubdtype(X[col].dtype, np.number):
            slider_inputs[col] = st.sidebar.slider(
                col,
                float(X[col].min()),
                float(X[col].max()),
                float(X[col].median())
            )

    # Create and align user input
    user_input_df = pd.DataFrame([slider_inputs])
    user_input_df = user_input_df.reindex(columns=X_train.columns, fill_value=0)

    # ---------- TRAIN MODELS ----------
    if st.button("🚀 Train & Predict"):
        st.info("Training models... please wait")

        # ===== QUANTILE MODELS =====
        qs = [0.1, 0.5, 0.9]
        models = {}

        for q in qs:
            model = lgb.LGBMRegressor(
                objective="quantile",
                alpha=q,
                n_estimators=600
```

```
        n_estimators=300,
        learning_rate=0.05,
        num_leaves=31,
        random_state=42
    )
    model.fit(X_train, y_train)
    models[q] = model

y_lo = models[0.1].predict(X_test)
y_med = models[0.5].predict(X_test)
y_hi = models[0.9].predict(X_test)

# ===== METRICS (TOP) =====
rmse = np.sqrt(mean_squared_error(y_test, y_med))
r2 = r2_score(y_test, y_med)
coverage = np.mean((y_test >= y_lo) & (y_test <= y_hi)) * 100
interval_width = np.mean(y_hi - y_lo)

st.success("Models Trained Successfully!")

c1, c2, c3, c4 = st.columns(4)
c1.metric("RMSE", f"{rmse:.3f}")
c2.metric("R²", f"{r2:.3f}")
c3.metric("Coverage %", f"{coverage:.1f}")
c4.metric("Interval Width", f"{interval_width:.3f}")

# ==========  🔹 REAL-TIME SINGLE PREDICTION ==========
st.write("## 🌾 Real-Time Yield Prediction (Your Inputs)")

user_pred_lo = models[0.1].predict(user_input_df)[0]
user_pred_med = models[0.5].predict(user_input_df)[0]
user_pred_hi = models[0.9].predict(user_input_df)[0]

st.metric("Predicted Yield (Median)", f"{user_pred_med:.2f} MT/ha")
st.write(f"Uncertainty Range: **{user_pred_lo:.2f} – {user_pred_hi:.2f} MT/ha**")

# ============================================================
# ✅ ALL PLOTS IN ONE SINGLE SCROLLING FRAME START HERE
# ============================================================

st.write("## 📊 ALL PROJECT VISUALS (CONSECUTIVE)")

# ===== PLOT 1: ACTUAL vs PREDICTED =====
st.write("### 1️⃣ Actual vs Predicted")

fig1, ax1 = plt.subplots(figsize=(7,7))
ax1.scatter(y_test, y_med, alpha=0.6)
ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
ax1.set_xlabel("Actual Yield")
ax1.set_ylabel("Predicted Yield")
ax1.set_title("Actual vs Predicted")
st.pyplot(fig1)

# ===== PLOT 2: UNCERTAINTY BAND =====
st.write("### 2️⃣ Uncertainty Band (10–90%)")

fig2, ax2 = plt.subplots(figsize=(7,7))
ax2.scatter(y_test, y_med, alpha=0.6)
ax2.fill_between(np.sort(y_test),
                 np.sort(y_lo),
                 np.sort(y_hi),
                 alpha=0.3)
ax2.set_title("Prediction with Uncertainty Interval")
st.pyplot(fig2)

# ===== PLOT 3: EVT TAIL RISK =====
st.write("### 3️⃣ Extreme Tail Risk (EVT)")

residuals = y_test - y_med
losses = -residuals[residuals < 0]

shape, loc, scale = gev.fit(losses)
extreme_loss = gev.ppf(0.99, shape, loc=loc, scale=scale)

x = np.linspace(0, max(losses.max(), extreme_loss)*1.1, 300)
pdf = gev.pdf(x, shape, loc=loc, scale=scale)

fig3, ax3 = plt.subplots(figsize=(7,7))
```

```
ax3.plot(x, pdf)
ax3.axvline(extreme_loss, linestyle="--", color="red")
ax3.set_title("EVT Tail Risk Curve")
st.pyplot(fig3)

st.write(f"99th Percentile Loss: {extreme_loss:.2f} MT/ha")

# ===== PLOT 4: FEATURE IMPORTANCE =====
st.write("### 4️⃣ Top 10 Important Features")

imp_gain = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": models[0.5].feature_importances_
}).sort_values("Importance", ascending=False).head(10)

fig4, ax4 = plt.subplots(figsize=(8,6))
sns.barplot(data=imp_gain, y="Feature", x="Importance", ax=ax4)
st.pyplot(fig4)

# ===== PLOT 5: SHAP EXPLANATION (USER INPUT) =====
st.write("### 5️⃣ SHAP Explanation for Your Input")

explainer = shap.TreeExplainer(models[0.5])
shap_values_user = explainer(user_input_df)

fig5, ax5 = plt.subplots()
shap.waterfall_plot(shap_values_user[0], show=False)
st.pyplot(fig5)

st.success("✅ All plots rendered consecutively in one frame.")
```

```
Overwriting app.py
```

```
!streamlit run app.py --server.port 8501 --server.address 0.0.0.0 &>/content/logs.txt &
```

```
from pyngrok import ngrok
ngrok.kill()
public_url = ngrok.connect(8501)
public_url
```

```
<NgrokTunnel: "https://inculpable-palma-selenic.ngrok-free.dev" -> "http://localhost:8501">
```